Week 6: Web Security & Vulnerability Testing (VAPT Basics)

Task 1: Study OWASP Top 10

1. Broken Access Control - Maintaining its position at #1 in the Top Ten, 100% of the applications tested were found to have some form of broken access control.

2. Security Misconfiguration - Moving up from #5 in the previous edition, 100% of the applications tested were found to have some form of misconfiguration, with an average incidence rate of 3.00%, and over 719k occurrences of a Common Weakness Enumeration (CWE) in this risk category.

3. Software Supply Chain Failures - This was top-ranked in the Top 10 community survey with exactly 50% respondents ranking it #1.

4. Cryptographic Failures - Sensitive information is inadequately protected through encryption or other cryptographic controls.

5. Injection - Untrusted input is interpreted as commands or queries, as in SQL Injection.

6. Insecure Design - Security weaknesses arise from missing or ineffective security controls in the application's design.

7. Authentication Failures - Weak identity verification or session handling allows unauthorized access.

8. Software or Data Integrity Failures - Software updates, code, or data are not adequately verified for integrity.

9. Security Logging and Alerting Failures - Security events are not properly recorded, monitored, or alerted on.

10. Mishandling of Exceptional Conditions - Errors and unusual conditions are handled insecurely, potentially creating vulnerabilities.

A. **SQL Injection (SQLi)** - SQL Injection occurs when an application handles user input unsafely, allowing it to alter database queries.
Example: A vulnerable login page may allow specially crafted input to change the logic of a database query.

Impact:

- Unauthorized database access.
- Exposure of sensitive information.
- Modification or deletion of records.
- Potential authentication bypass.

Prevention:

- Use parameterized queries and prepared statements.
- Validate input appropriately.
- Apply least-privilege database permissions.
- Avoid exposing detailed database errors.

B. Cross-Site Scripting (XSS) - XSS occurs when a website allows untrusted content to execute as code in another user's browser.

Types:

1. **Stored XSS:** Malicious content is stored by the application and displayed to users.
2. **Reflected XSS:** Malicious input is included in an immediate response.
3. **DOM-based XSS:** Unsafe client-side JavaScript processes untrusted data.

Impact:

- Manipulation of web page content.
- Phishing attacks.
- Unauthorized actions within a user's browser.
- Exposure of information accessible to malicious scripts.

Prevention:

- Apply context-aware output encoding.
- Use safe DOM manipulation.
- Validate input.

- Implement a suitable Content Security Policy (CSP).

C. Broken Authentication - Authentication failures occur when an application does not adequately verify user identities or securely manage authenticated sessions.

Examples:

- Weak password policies.
- Unlimited login attempts.

Impact:

- Unauthorized access.
- Exposure of confidential information.

Prevention:

- Implement multi-factor authentication.
- Use secure password-hashing algorithms.
- Apply login rate limiting.

D. Security Misconfiguration - Security misconfiguration occurs when an application, server, database, or cloud service is configured insecurely.

Examples:

- Default credentials remain unchanged.
- Unnecessary services are enabled.

Impact:

- Unauthorized access.
- Data exposure.
- Increased attack surface.

Prevention:

- Change default credentials.
- Disable unnecessary services.
- Apply security patches.

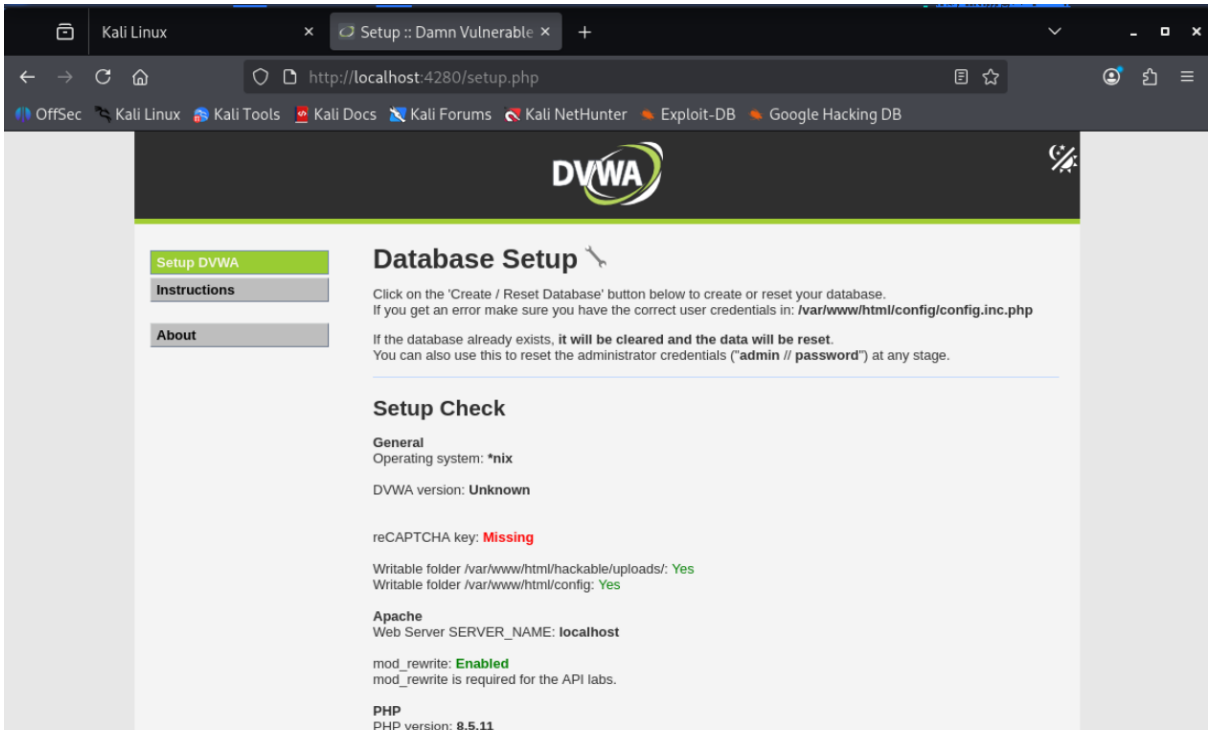
Task 2: Install & Setup Tools

- Burp Suite (Community Edition):
  - Open the kali terminal and run: sudo apt update
  - Sudo apt install burpsuite -y
  - Then launch it: burpsuite

- Install OWASP ZAP on Kali Linux:
  - https://www.zaproxy.org/download/

- Set up DVWA using Docker on Kali Linux
  - In kali terminal: sudo apt update
    sudo apt install docker.io -y
  - Then enable docker and check version: sudo systemctl enable --now docker
    docker –version
  - We install Docker Compose because DVWA needs multiple services to run together: the DVWA web application and its database. To install it:
  - sudo apt install docker-compose
  - Then start DVWA:
    cd ~/Desktop
    git clone https://github.com/digininja/DVWA.git
    cd DVWA
  - Then start the application: sudo docker-compose up -d
  - Check the containers: sudo docker-compose ps
  - **Open DVWA**
    Open Firefox and visit:
    http://localhost:4280

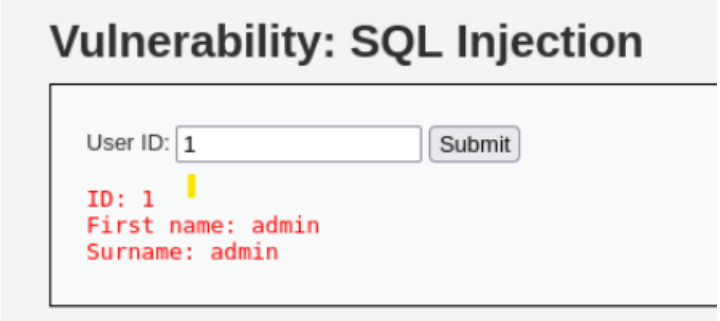
```
┌──(nehawalmiki㉿kali)-[~/Desktop/DVWA]
└─$ sudo docker-compose up -d
[sudo] password for nehawalmiki:
[+] Running 29/29
 ✔ db Pulled
   ✔ 98c4455a9898 Pull complete
   ✔ c34808af6b3e Pull complete
   ✔ 37f986522545 Pull complete
   ✔ 3e968919cb92 Pull complete
   ✔ e7908f300846 Pull complete
   ✔ 4ef363dc51e7 Pull complete
   ✔ 59013e7d205a Pull complete
   ✔ d626f8bd756e Pull complete
 ✔ dvwa Pulled
   ✔ ecc510c1e359 Pull complete
   ✔ b06d38994c79 Pull complete
   ✔ 98ec16867954 Pull complete
   ✔ fadac184c029 Pull complete
   ✔ e37af3e95c47 Pull complete
   ✔ fa171a020838 Pull complete
   ✔ f4527b7aef5d Pull complete
   ✔ 3abeca9d74cb Pull complete
   ✔ 6c1de721bd73 Pull complete
   ✔ 0dd2ad9d76f9 Pull complete
   ✔ e10234afeede Pull complete
   ✔ 4814c1f6b2f7 Pull complete
   ✔ 8f4682996e77 Pull complete
   ✔ 4f4fb700ef54 Pull complete
   ✔ 5c85b548e453 Pull complete
   ✔ c86e106a1086 Pull complete
   ✔ 8551bd71971b Pull complete
   ✔ 42a59ae6b1ad Pull complete
   ✔ 23d9e9e08523 Pull complete
[+] Running 4/4
 ✔ Network dvwa_dvwa       Created
 ✔ Volume dvwa_dvwa        Created
 ✔ Container dvwa-db-1     Started
 ✔ Container dvwa-dvwa-1   Started
```



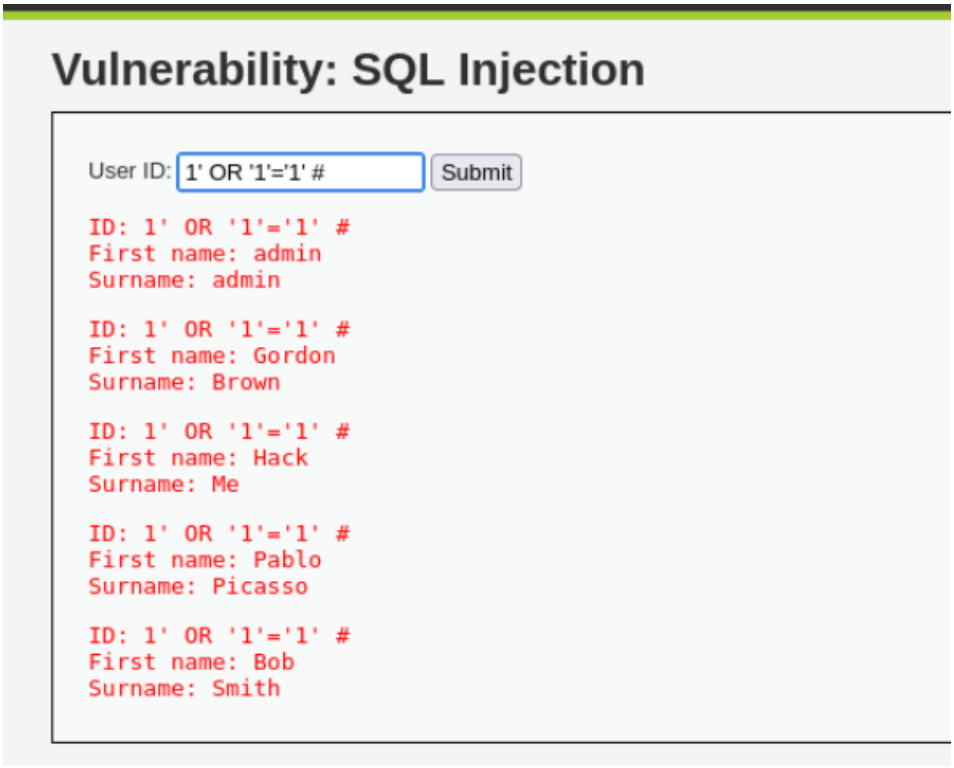
Task 3: Perform Testing (DVWA)

- Test SQL Injection

Enter a normal value



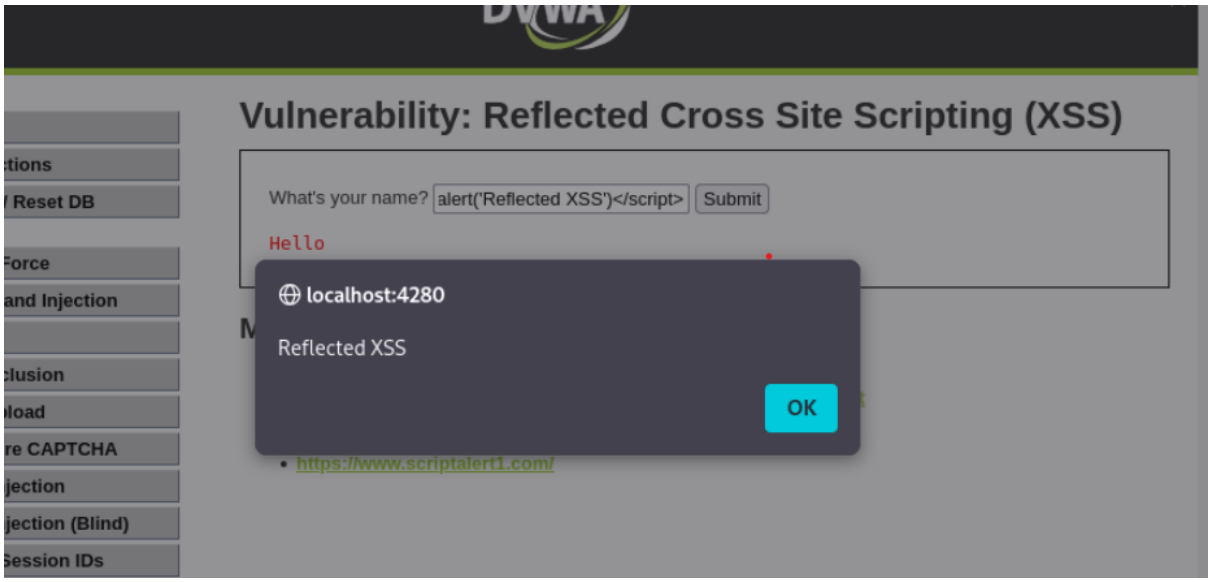
Test a basic SQL injection condition in the local lab by entering:



Result: The SQL injection test was successful. Normally, entering user ID 1 should display only one user's details. However, the injected condition ' OR '1'='1' is always true, causing the application to display multiple user records. The # symbol comments out the remaining SQL query in MySQL. This shows that the application does not safely handle user input.

- Test XSS (Stored & Reflected)

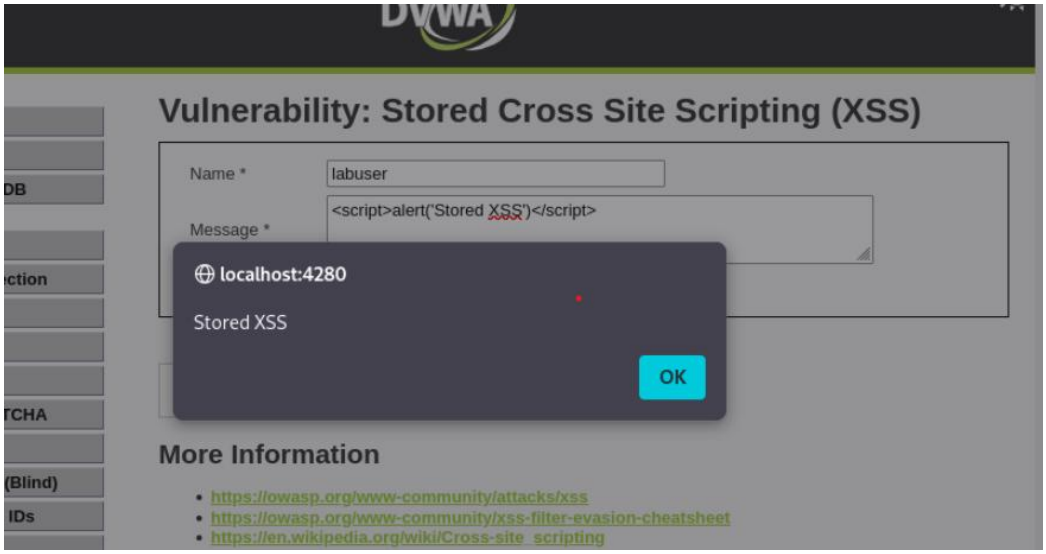
1. Open XSS (Reflected) from the DVWA menu.
2. Enter <script>alert('Reflected XSS')</script> and submit
3. Check whether the value appears in the response page.



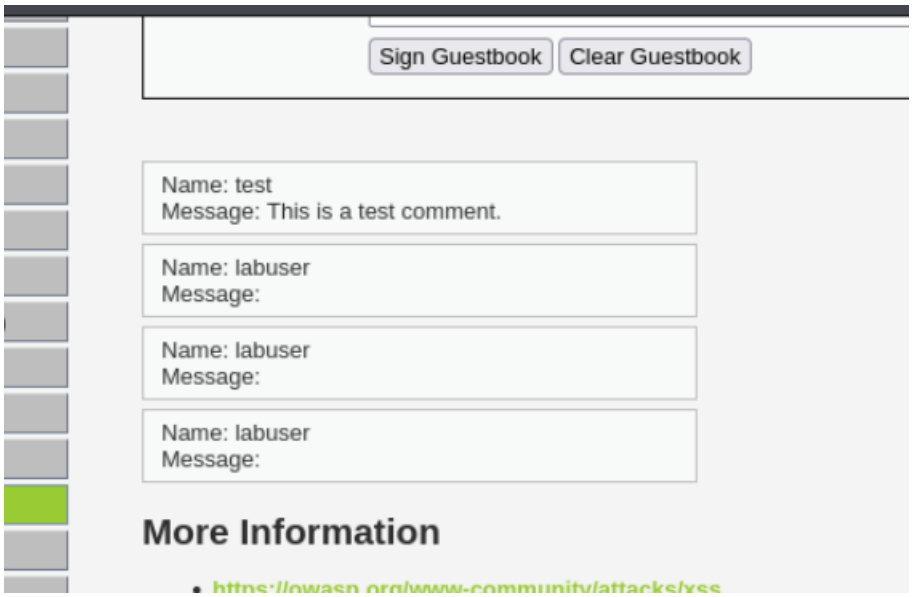
Browser alert dialogue box successfully generated from localhost:4280 displaying the string **"Reflected XSS"**. This confirms the presence of a high-severity flaw.

- Test Stored XSS

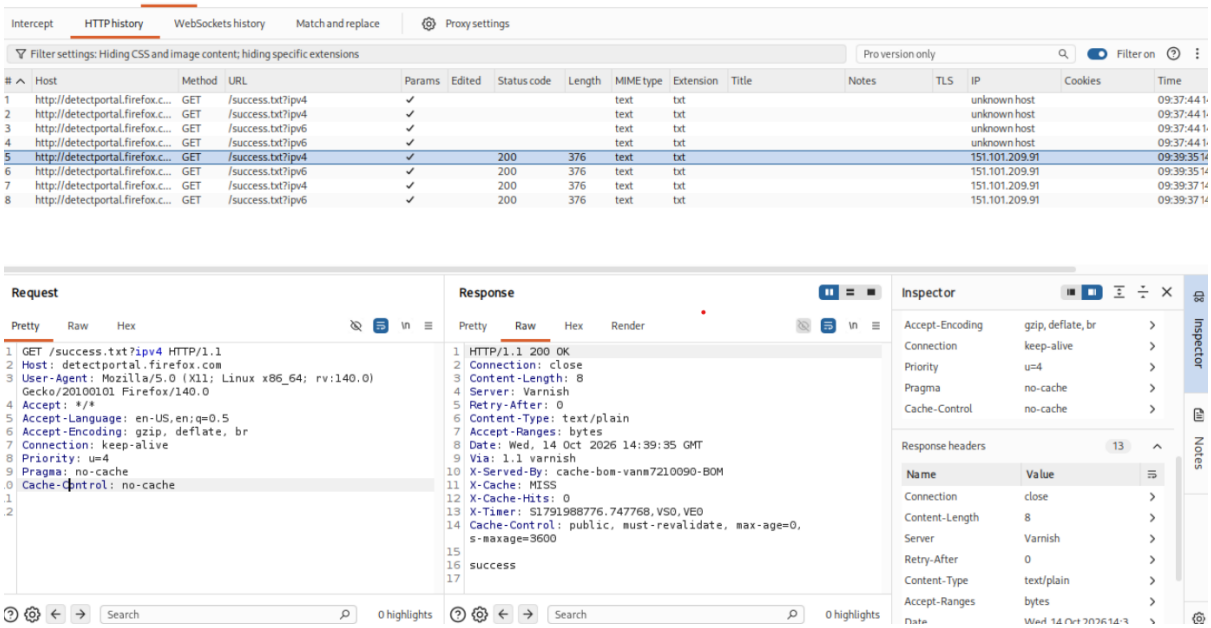
1. Open XSS (Stored) from the DVWA menu.
2. Enter a name such as labuser.
3. Enter <script>alert('Stored XSS')</script> and submit
4. Verify that the message appears in the guestbook or message list.



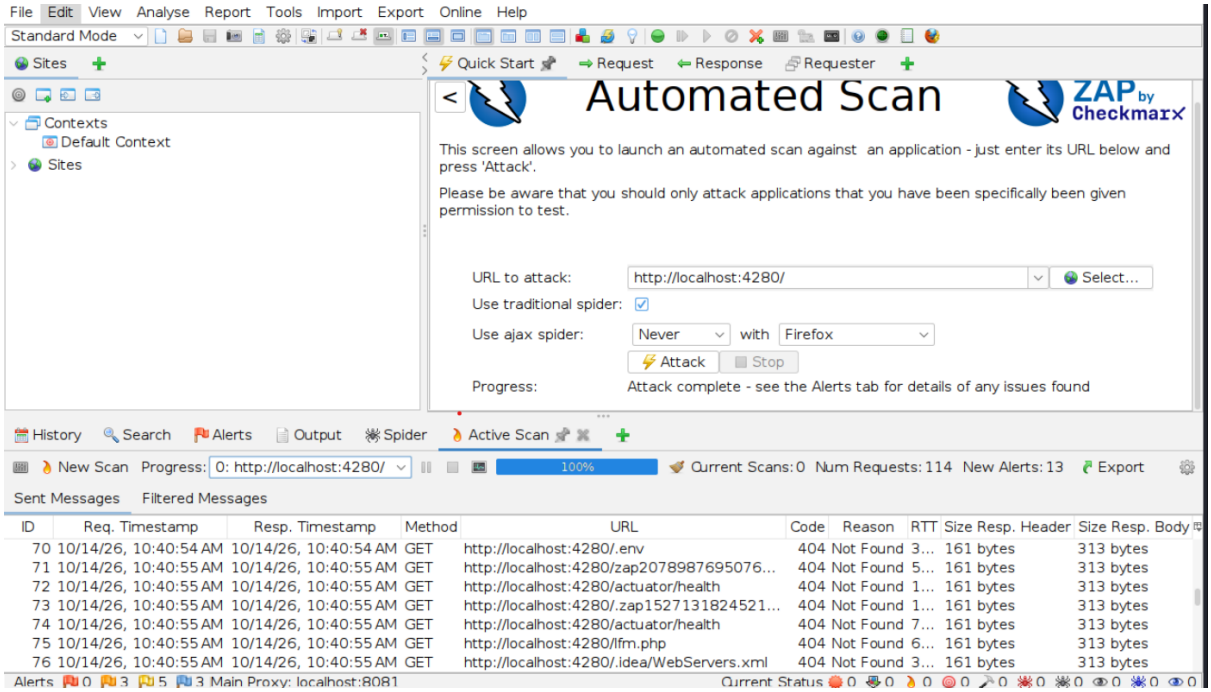
When I click the button to submit, the website saves this code directly into its database. Because the website doesn't clean the text, it immediately runs the code, popping up the grey box that says **"Stored XSS"**.



The text isn't actually missing. The text *is* the hidden code. Every single time this page loads, the web browser reads that hidden code and executes it behind the scenes instead of displaying it as text.



Task 4: Scan Using OWASP ZAP



1. On the Quick Start tab, click Automated Scan.
2. In the URL to attack field, enter: http://localhost:4280/
3. Click Attack and wait for ZAP to finish crawling and scanning the site.

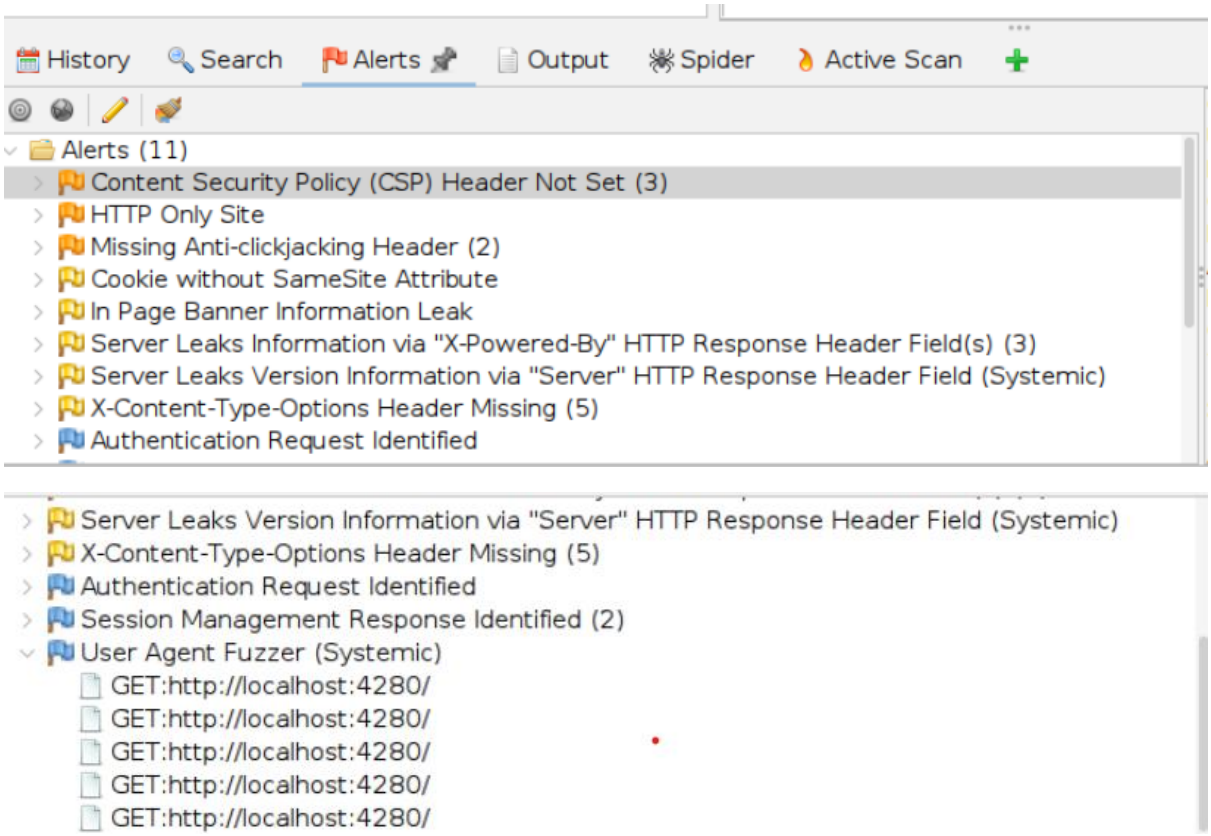
In total, ZAP sent 114 requests and flagged 13 vulnerabilities

🟠 Medium Risk Alerts

- **HTTP Only Site:** The application runs over unencrypted HTTP instead of secure HTTPS. All data passed between the user and the application—including passwords and session tokens—is vulnerable to interception via **Man-in-the-Middle (MitM)** attacks.
- **Missing Anti-clickjacking Header (2 instances):** The server lacks X-Frame-Options or CSP frame-ancestor's directives. This allows malicious sites to load your application inside an invisible <iframe>, tricking users into clicking buttons they did not intend to click.

🟡 Low Risk Alerts

- In Page Banner Information Leak: The application exposes software, system version names, or engineering details directly inside the visible HTML source code, assisting attackers with targeted recon.



Task 5: Project Report

- **Introduction to VAPT**

Vulnerability Assessment and Penetration Testing (VAPT) is a cybersecurity process used to identify, evaluate, and validate security weaknesses in systems, networks, and applications.

- **Tools used**

- **DVWA**
  Damn Vulnerable Web Application is an intentionally insecure web application used for learning web application security testing.

- **XAMPP**
  XAMPP is a local development environment that includes components such as Apache, MariaDB, PHP, and Perl.
  Apache serves the web application, PHP executes application code, and the database stores application data.

- **Burp Suite Community Edition**
  Burp Suite is a web application security testing tool. Its proxy functionality allows a tester to inspect HTTP requests and responses between a browser and a web application.

- **OWASP ZAP**
  OWASP Zed Attack Proxy (ZAP) is an open-source web application security testing tool.

- **Kali Linux**
  Kali Linux is a Linux distribution designed for penetration testing and security assessments. It provides a platform for running security tools, including OWASP ZAP.

- **Testing process**

SQL Injection Test -

Testing Procedure

1. Open DVWA and log in.
2. Navigate to the SQL Injection module.
3. Enter a normal test value and record the expected result.
4. Enter a controlled SQL Injection test input.

5. Submit the input.
6. Compare the response with the normal request.
7. Capture the relevant request and response in Burp Suite.
8. Record the actual output and evidence.

Cross-Site Scripting Testing

Testing Procedure:

1. Open the Stored XSS module.
2. Submit a normal test value first.
3. In the authorized lab, enter the same harmless proof-of-concept if the module permits it.
4. Submit the input.
5. Reload or revisit the page.
6. Observe whether the content persists and executes.
7. Capture the submission and subsequent page response.

OWASP ZAP Automated Scan

Testing Procedure:

1. Launch OWASP ZAP in Kali Linux.
2. Select Quick Start.
3. Open Automated Scan.
4. Enter the DVWA URL.
5. Click Attack.
6. Allow ZAP to discover and test accessible pages.
7. Review the Alerts tab.
8. Inspect alert evidence and risk levels.
9. Export the results in HTML format.

- **Findings & Risk Levels**

We were able to find alerts using ZAP.
Risk level was high, medium and low

- **Mitigation methods**

SQL Injection Mitigation

Recommended controls:

1. Use prepared statements and parameterized queries.
2. Avoid concatenating untrusted input into SQL commands.

XSS Mitigation
Recommended controls:

1. Use context-aware output encoding.
2. Use safe templating and DOM methods.

Authentication Mitigation
Recommended controls include:

- Secure password hashing.
- Appropriate multi-factor authentication.
